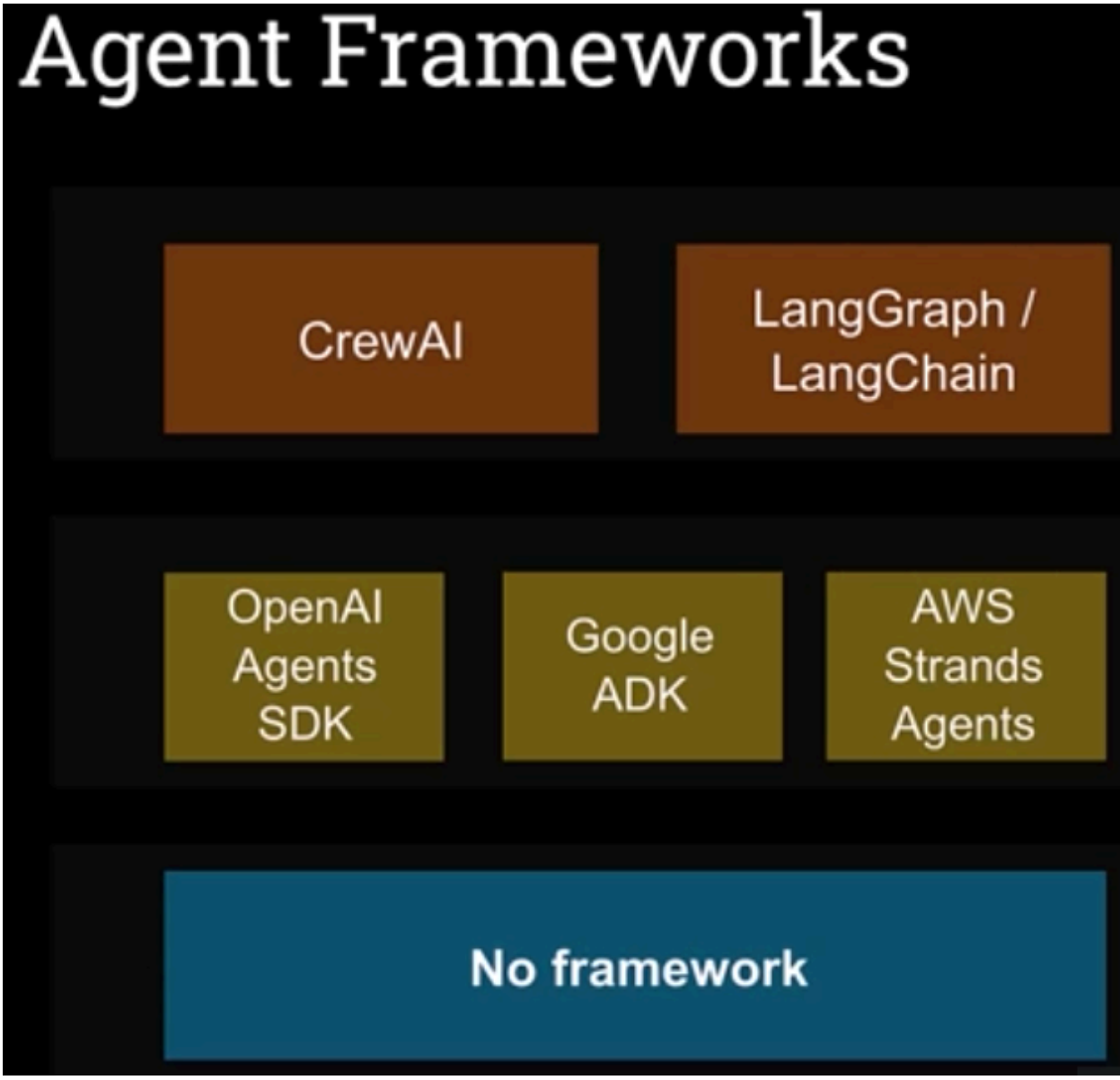
LLM autonomy and tool integration

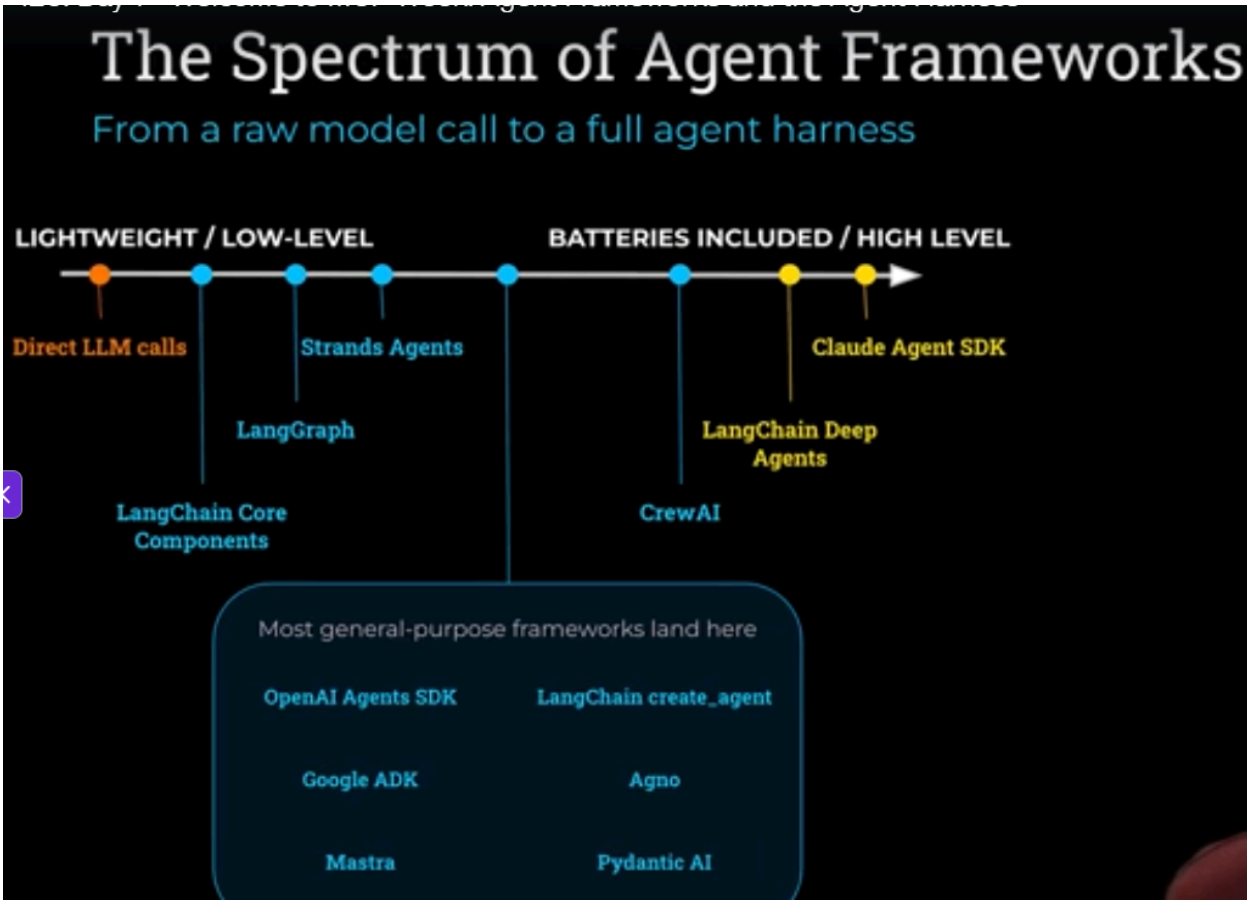
- What is an agent?
    - Can mean a lot of different things when people say “agent”
    - **Definition #1** by Sam Altman: “Agents are AI systems that can do work for you independently”
    - **Definitions #2**: Huggingface/Anthropic definition
        - AI agents are programs/systems where LLM outputs control the workflow
            - They decide what tasks are carried out and in what sequence
            - They decide how to go about a task, not the code - prompt based rather than code based
            - Use the Agent output to control the workflow
            - In reality LLMs don’t carry out actions, they generate tokens that we interpret either in code or using another LLM
            - An agent is our code that interprets the tokens coming from an LLM
    - **Definition #3**: agent = LLM with tools in a loop to achieve a goal
        - Agent = LLM that has tools to achieve a goal in a loop
        - An agent carries out multiple sub-tasks in a loop with tools until it has reached an outcome to the original problem
        - An LLM call is not agentic yet
            - **LLM call != agent**
            - An LLM call takes an input with tokens and produces an output with tokens
            - We have to add tools, prompts, code, agents to the mix and give the agent autonomy in order to reach a higher level of “agentic-ness”
            - Our code turns vanilla LLM calls into AI agents
        - …or an Agent = LLM with a harness, with harness becoming the word to describe all the key ingredients like goals, loops and tools
        - It’s the harness that turns an LLM into an AI agent
        - We can build the harness ourselves using the basic building blocks in code, like the OpenAI Agents SDK or we can use frameworks such as CrewAI, LangGraph

- A simplistic look at frameworks:



- A simplistic look at frameworks (see image)
  - The agent frameworks can also be put on a continuum from a raw model call to a full agent harness, from “lightweight / low-level” to “batteries included / high level”
  - Low-level: more dev work but also more control
  - High-level: less dev work but also less control, and a more directed, opinionated way of building agentic workflows
    - Less control: harder to debug as well
  - Claude Agent SDK: not really a framework but very high level and can only work with the agent that comes with Claude Code
    - Programmatically drive Claude Code, a very specific Anthropic product
    - Build AI agents that autonomously read files, run commands, search the web, edit code and more. The Agent SDK gives you the same tools,

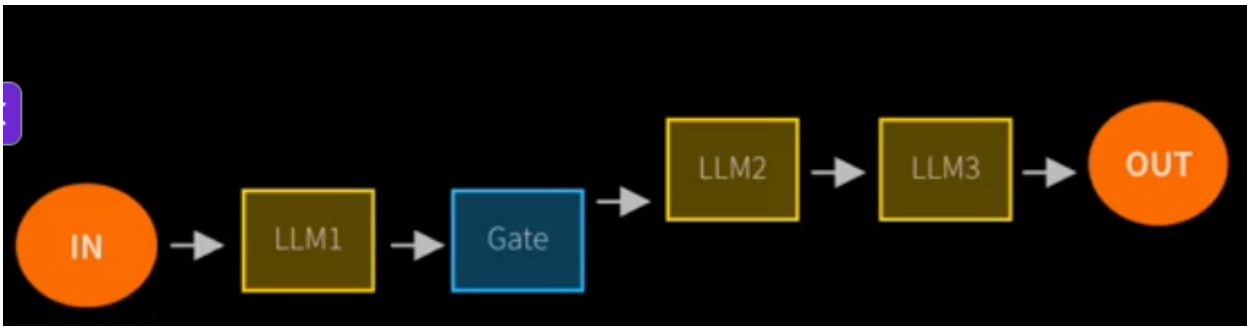
agent loop and context management that power Claude Code, programmable in Python and Typescipt

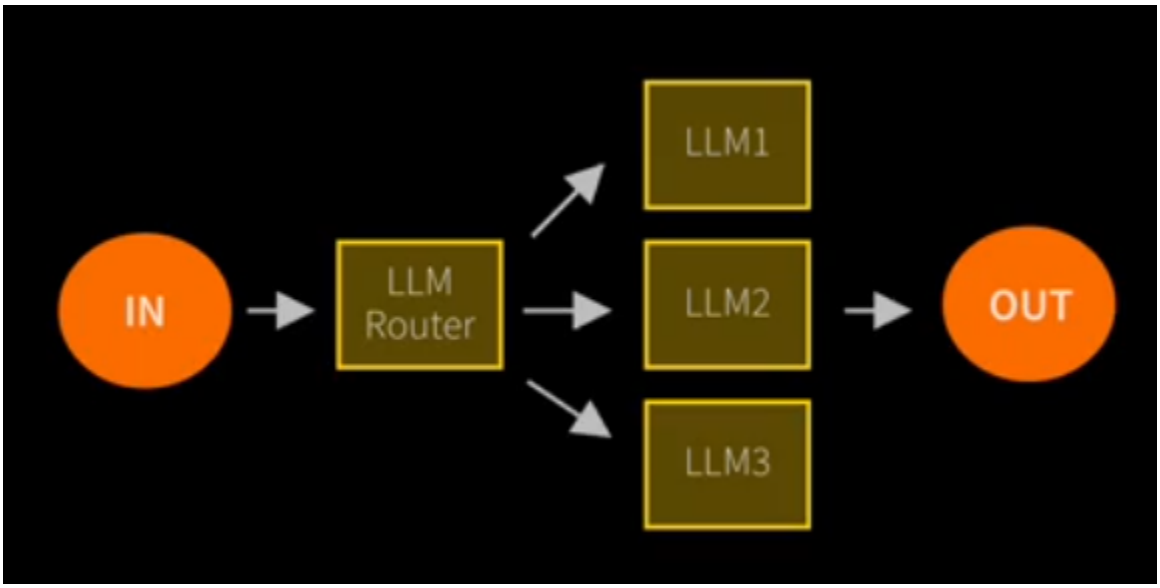


- Hallmarks of agentic solution
  - Multiple LLM calls
  - LLMs with the ability to use tools
    - Often a litmus test to check whether a process is agentic or not
  - An environment where LLMs interact
    - E.g. orchestration
  - A planner to coordinate activities
  - Autonomy
    - Captures the essence of agentic AI
    - Give LLM the control to “choose its own adventure” in order to solve a task
    - E.g. give a task to an agent and then let it ask another agent to evaluate its initial output
- Important link: [Building Effective AI Agents \ Anthropic]
  - Two types of agent system
    - **Workflows**
      - Are systems where LLMs and tools are orchestrated through predefined code paths
    - **Agents**

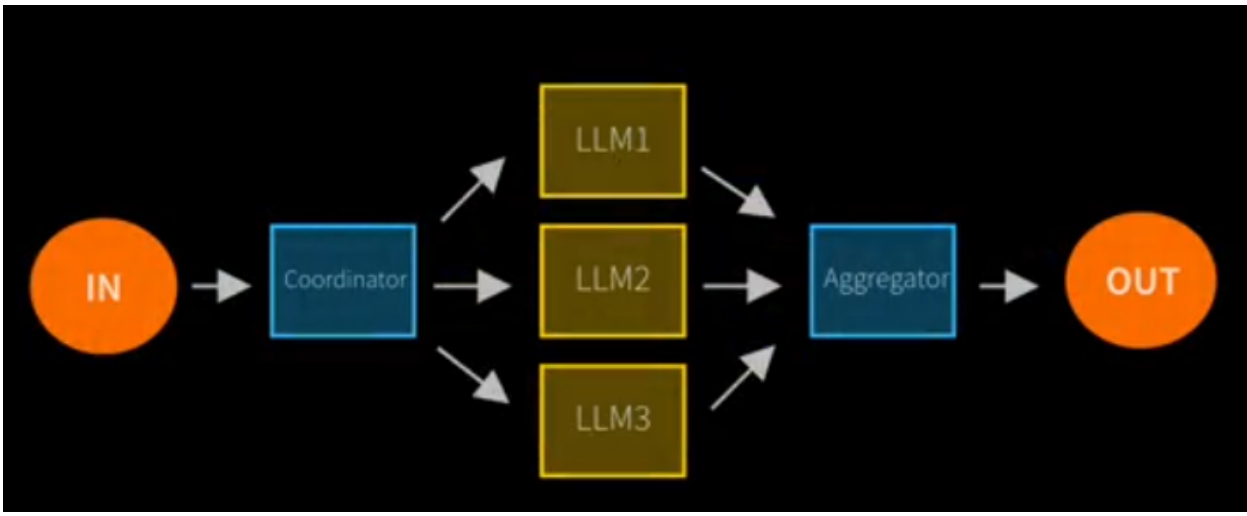
        - Are systems where LLMs dynamically direct their own processes and tool usage, maintaining control over how they accomplish tasks
- Agents vs workflows
    - Workflows are systems where LLMs and tools are orchestrated through predefined code paths
        - E.g. a deep research workflow where you ask a question, the LLM comes back with clarifying questions, processes your responses, searches the web and churns out an output to the user
    - Agents are dynamic systems where LLMs direct their own processes and tool usage, maintaining control over how they accomplish tasks
        - E.g. booking agents that can go out and search for available tickets or seats in a restaurant a make bookings on your behalf
    - In practice the boundary is not always clear and well defined
        - An AI app can show traits of both
    - Real business apps generally favour workflows as they are more deterministic, close-ended, repeatable and are easier to debug than agents
        - Can sprinkle with agents along the way with a low level of freedom (guardrails apply!)
- Agentic AI involves building a harness around LLMs, crafting the inputs and interpreting the outputs, in a loop, so that they can “use tools” and “be autonomous”
- Agent frameworks are helper code that make it faster to implement these techniques but they are not required
    - We can write agentic AI without any framework if we don’t mind writing a lot of JSON and boilerplate code

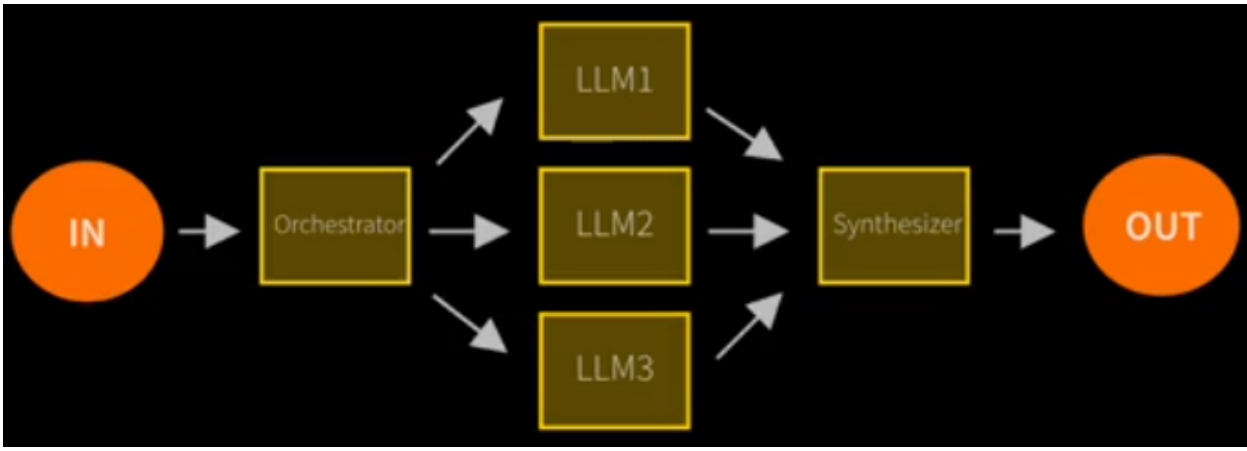
Agentic workflows and design patterns

- Based on Anthropic blog post [Building Effective AI Agents \ Anthropic]
- Agentic design patterns are not as rigid or well-defined and mature as design patterns in sw engineering
    - It's not like a problem has a well defined agentic design pattern to follow
    - Can combine design patterns without clear boundaries
    - Also, the below patterns come in many shapes and forms in real life business apps, they don't always look the same and can be challenging to distinguish
- **Prompt chaining**
    - Decompose a bigger task into fixed sub-tasks
    - 

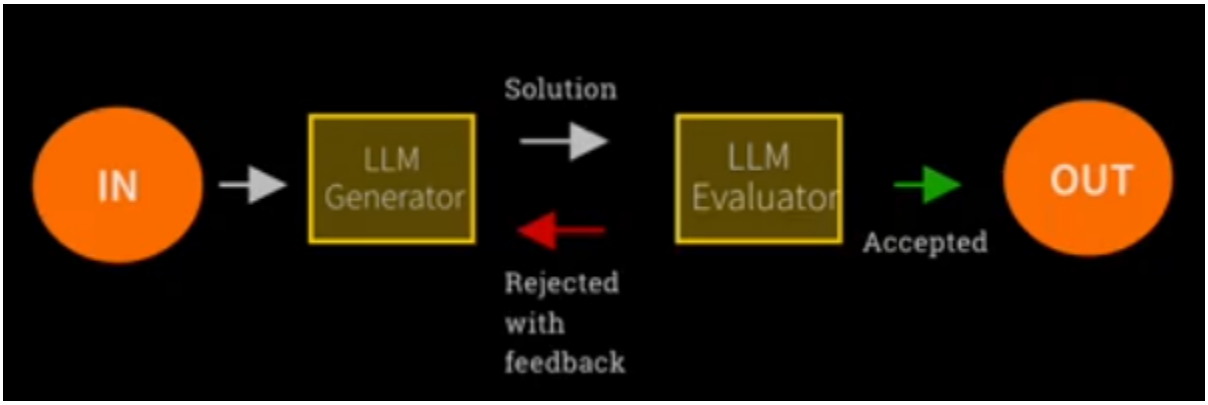
    - Yellow: calls to models
    - Blue: some code, but optional
    - LLM carries out a task, then calls a bit of code then call another LLM, a third and so on until we get to the output
    - A sequence of well-defined tasks in order to get from input to output
    - Freedom given to the LLMs in each yellow step can vary
    - So there **is** at least a certain level of autonomy to each LLM step
- **Routing**
    - Direct an input into a specialised sub-task, ensuring separation of concerns
    - 

    - The router decides which LLM carries out the task based on the input
    - The router is like a classifying process to decide the owner of the overall task

    - Router typically has most autonomy out of all LLMs involved
    - E.g. a customer service system can have a router that decides which LLM to call: technical support? Or business support? This is up to the router to decide based on the incoming initial prompt
    - Could also solve the problem with a single LLM but it's often best to let specialised LLMs take care of the actual matter once the router made up its mind: separation of concerns in AI
- **Parallelisation**
    - Breaking down tasks and running multiple subtasks concurrently to arrive at a goal
    - 

    - Blue: code, not an LLM call
        - Code will do the coordination, calls three LLMs to carry out the tasks in parallel
        - Can be the same task three times
        - Each LLM output is aggregated in code and we get to the output
- **Orchestrator-worker**
    - Complex tasks are broken down dynamically and combined
    - 

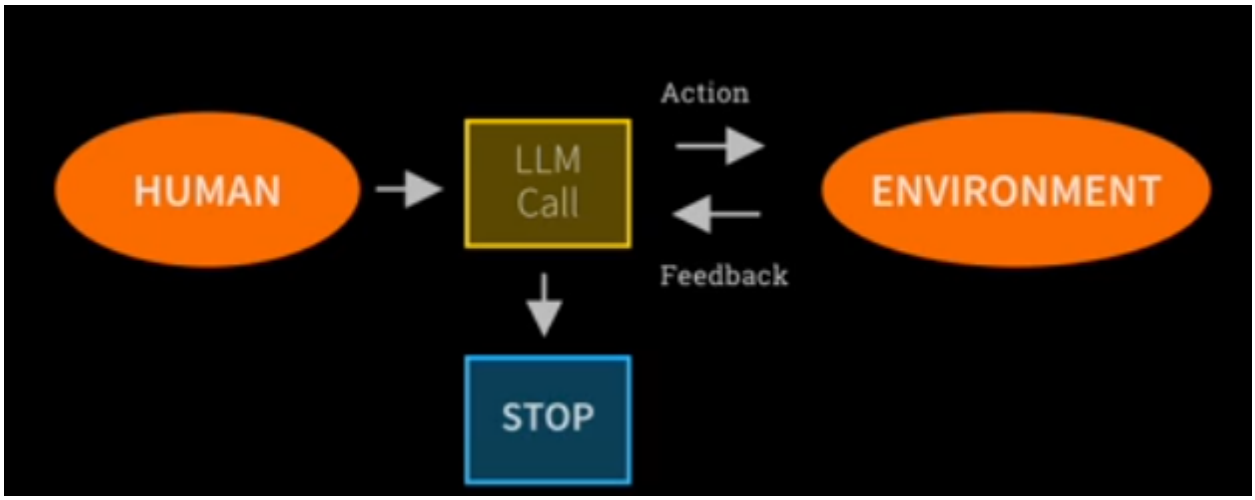
    - A challenging task is broken down into sub-tasks which are then recombined

    - Similar to parallelisation but here no code is involved, LLM does the orchestration and the combination of results, much more dynamic system than the one before this
    - First it looks like there's no control, everything is carried out by LLMs but the orchestrator and synthesizer LLMs are also controllers, but they are LLM based
- **Evaluator-optimiser**
    - Also called "validators", "evaluators", "validation pattern" etc.
    - LLM output is validated by another
    - 

    - Very common pattern and gives most autonomy to the LLMs
    - LLM generator comes up with "something" which is evaluated by the evaluator and then either rejects or accepts it
        - Rejection must be accompanied with a feedback
        - Validation is often a must for production-level AI systems
    - This is a pattern with a "judge", called the evaluator
- Which framework to choose for your project?
    - In fact it doesn't really matter, there's no much difference between the various available frameworks
    - Pick the framework that suits your style and the skills of your team
    - If you e.g. CrewAI with its separation of YAML and python code + all the batteries included then go for it
    - If you prefer the simplicity of OpenAI agents SDK then go with that
    - You can essentially achieve the same things in the various frameworks, this is by far not the most critical choice you have to make it you want to build an agentic workflow
- Here are the things that matters much more than selecting a specific framework
    - Start with the problem, not the solution
        - Focus on the business problem first and then decide if it can solved via an agentic workflow
        - Don't begin with "I want an agent that does…", that's the wrong end to begin with
    - Have a metric to evaluate success
        - You must be able to measure the outcome of the solution
        - This can be really difficult to implement, a real measure to help you make sound judgements

- Favour workflow over autonomy initially
  - Prefer orchestrating with code at first before considering LLM-based orchestration with tools and a lot of decision power
  - Good to have more control at first, also helps with building metrics
  - A lot of successful AI software comes with code-based workflows rather than true agentic AI like Claude code
  - Repeatable, easier to debug and track, easier to check in the logs what happened during execution
  - First come up with a resilient and bulletproof, code based workflow before experimenting with more autonomy and see if you get better outcomes or not
- Work bottom-up, not top-down
  - Don't tackle a huge problem with lots of agents, diagrams etc
  - Tackle a small piece of the problem first before turning to the large problem
  - Start small and work your way up
- Start simple then add more stuff
  - Similar to the previous point
  - Start with a single LLM call instead of building a large LLM architecture up front
  - Get your first LLM call working really well and then break it up if necessary into two LLM calls and iterate over that
  - Be driven by data and metrics and measure if you get better outcomes
- Start with large frontier models, then reduce
  - Starting with small models like GPT-5.3-nano is cheaper but the outcome is ambiguous and often frustrating
  - Start a large model at first, although it's more costly, stabilise your prompts and then try them with a smaller model
- Think context rather than memory
  - Think about the context that can be sent into the LLM instead of turning to memory up front
- Most problems are solved with prompts
  - Improve the prompts!
  - Many bugs and issues with the LLMs are often down to bad or unclear prompts
  - Try different prompts and measure the different outcomes
- Look at the traces
  - Verify that things are actually working as intended
  - Verify that your LLM actually uses the tools that you equipped it with and didn't just make up an answer
- Be a scientist besides a developer
  - There's no shortcut to R&D
  - Dig in, roll up your sleeves, experiment and you're going to get good outcomes

Agent patterns vs workflows

- Agents are not the same as workflows - at least not in Anthropic's definition
- More open-ended than workflows
- Has feedback loops
- No fixed path through the pattern, much more fluid than workflows
- Therefore tasks that agent patterns can take on are much more complex than the workflow ones
- However, it's less predictable
    - More concern about robustness, guardrails and predictability
    - More difficult to reach production-level solutions
- 

- The flow is also much looser than the ones describing agentic workflows
    - "Environment" is an outside world that can be interacted with
    - LLM can take action on the env
    - Env gives feedback to the LLM
    - Up to LLM to decide when to stop, it may run for many loops
    - outcome , duration etc. unclear and less deterministic than in workflow patterns
- Drawbacks
    - Unpredictable path
    - Unpredictable output
        - More uncertainty
    - Unpredictable costs and duration
- Mitigations
    - Monitor, monitor, monitor
        - Must have visibility into what's going on during the agents interacting to solve a task
    - Guardrails
        - Guardrails ensure your agents behave safely, consistently and within your intended boundaries

Risks of agent frameworks

- Risks
    - Unpredictable path
        - Difficult to predict exactly what an agent is going to do, how it's going to interpret the tokens
    - Unpredictable output
        - Linked to the previous point: we don't know what kind of output it's going to generate
        - They are much less deterministic than traditional software, sometimes not deterministic at all
    - Unpredictable costs
        - Therefore even costs are unpredictable
- Mitigations
    - Monitoring
        - Observability, evals
        - Both during development and then under production
        - Evals: evaluate the agentic AI system, e.g. evaluate its performance against a professional end goal, a commercial/business goal
        - E.g. if business goal = sell more, then measure whether the AI system is generating more leads and customer contacts, or how much more revenue the business is generating and how much of that can be linked to the AI system
        - You can also use another LLM as a judge (evaluator pattern) to evaluate how well your agentic system is performing
    - Guardrails
        - Set boundaries to the LLM, describe what it can and cannot do
        - Stay within boundaries!
        - Must build up a scaffolding around your agentic workflow to make sure it stays within its limits
        - E.g. use evaluators to make sure the output is professional and doesn't contain PII data
- Traps
    - Requirements that want to force an agentic solution because it's hyped
        - "I need an agent for my business"
        - Rather consider the business problem you're trying to solve, may not need any agentic solution at all
        - The solution may simply be an LLM call, a vanilla Gen AI LLM call
        - Start from the problem to solve, instead of pushing for a certain type of solution up front
        - Always keep in mind that LLMs are not intelligent or sentient beings - they are trained to generate output that's hopefully based on facts and is not too far from the truth, and is at least believable. However, "believeable" != "necessarily right". It is then up to us humans to evaluate whether the generated output fulfils the requirements and business goals

        - Building agents all over the place may end up in high costs and no actual problems solved
    - Treat agents as anthropomorphic beings
        - As if they are humans with feelings and brains
        - Don't feel the rush to to assign human-like responsibilities to agents
        - I want to build a trading platform with a "Trading manager", "market researcher", "trader" and produce an agent architecture with a diagram up front
        - LLMs are token generation engines, not human beings
        - Better approach:
            - Start simple
            - Start with one LLM and one prompt and see how far you come with that
            - One LLM, one prompt, one objective and measure the effectiveness of the LLM against the expected or required outcome: sales leads up by 5%? Complaints down by 5%?, something of a tangible outcome based on the first LLM call
            - Then iterate, break up initial solution into two agents and two or more prompts and see how well it performs compared to the simpler solution
            - You may end up with a large architecture with a lot of boxes and agent roles, but you must get there gradually, not up front
            - Don't try to mimic human behaviour, like humans can be traders, sales assistants, researchers etc.
    - Blame bugs and bad output on hallucinations
        - Hallucinations do happen and it's up to YOU as the AI dev to make sure that hallucinations don't sip into the business output
        - Make sure there are guardrails, evaluations, logs etc. are all up to the AI developer
        - LLMs are trained to produce tokens based on complex statistics but that's their only goal, that's why they exist
        - Don't blame hallucinations, prepare for them in code!

Agentic engineer

- Who is an agentic engineer?
    - Could be someone who uses agents to engineer solutions
        - E.g. use Codex/Claude to produce a code-based solution
    - Could also be someone who engineers agents
        - This interpretation is the more prevalent case
    - …therefore always be specific when using these terms to describe a role/job
        - AI coder: an engineer skilled at using coding agents
        - AI engineer: skilled at building agents with code

Agent landscape

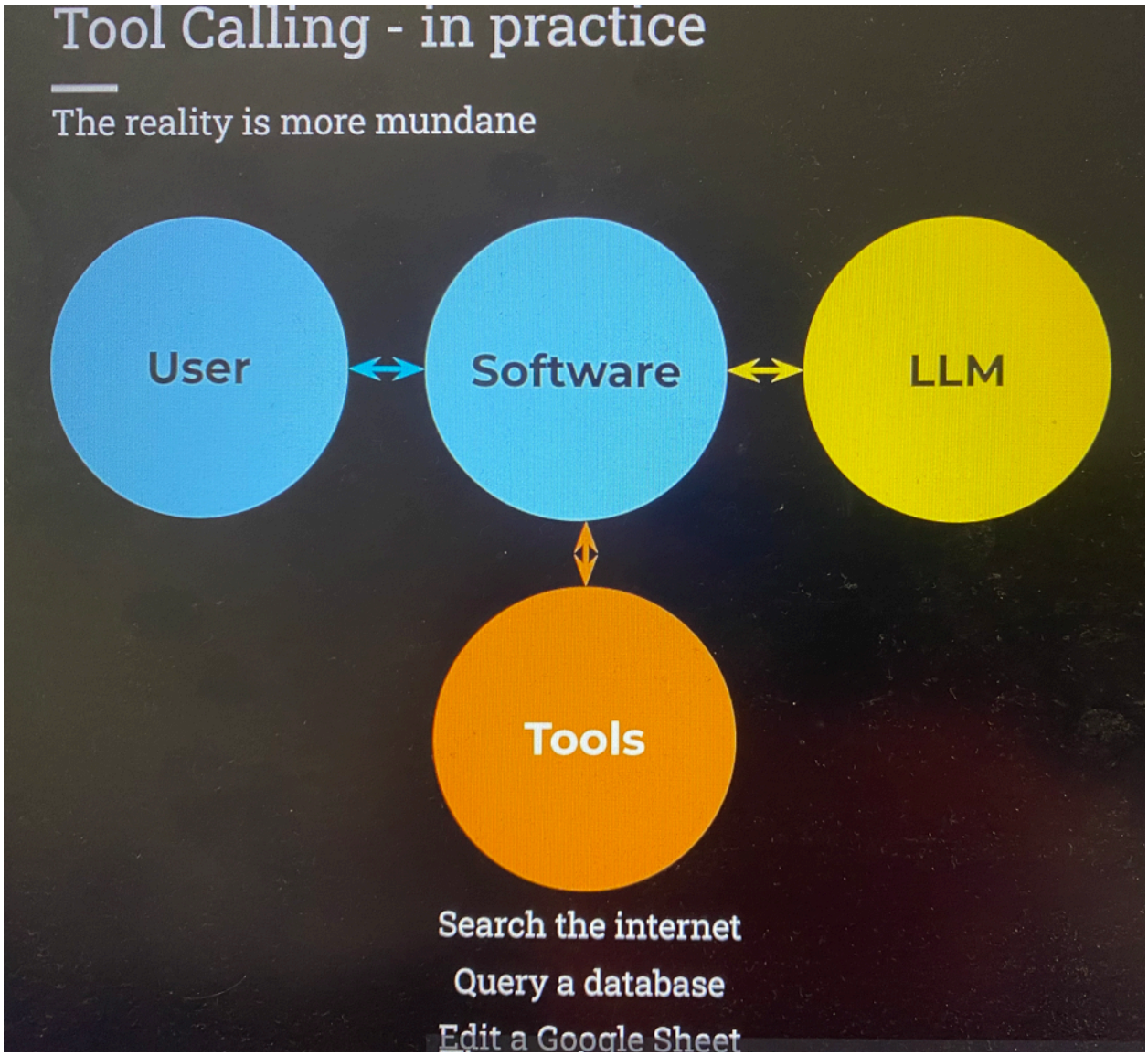
- The agentic AI field is overhyped and largely immature
- Lots of commercial offerings are overlapping
- Builders
    - Non-technical people can **build** AI agents, often drag-and-drop
    - N8n, ElevenLabs, OpenAI Agent Builder, CrewAI Studio
- Products
    - Non-technical people can **use** AI agents
    - Claude Code, Cowork, OpenClaw, Claude Design
- Runtimes
    - Technical people can **execute** AI agents
    - AWS Lambda, AWS Bedrock AgentCore, Claude Managed Agents, Vertex AI Agent Engine
- Frameworks
    - Technical people to **develop** AI agents
    - OpenAI Agents SDK, CrewAI, LangGraph/LangChain, Google ADK
    - Helper code that makes it easier to create agents based on LLM calls
    - Take care of stuff that otherwise would be too repetitive
    - Similar to abstraction layers because they hide some of the common functions you might have to do again and again
    - Abstraction frameworks to make agent development more streamlined
    - They take care of multiple things
        - Orchestration: chaining together LLM calls
        - Tool calling: equipping an LLM with abilities including with MCP, instead of the dev making lots of JSON-based function calls to describe tools
        - Construct LLM inputs and interpreting outputs, including a technique called structured output, in order to add agentic capability of deciding the course of action
    - These features of frameworks allow you to easily code an agent loop - an LLM in a loop with tools to achieve a goal
    - You can build agentic systems without any framework at all

Tools

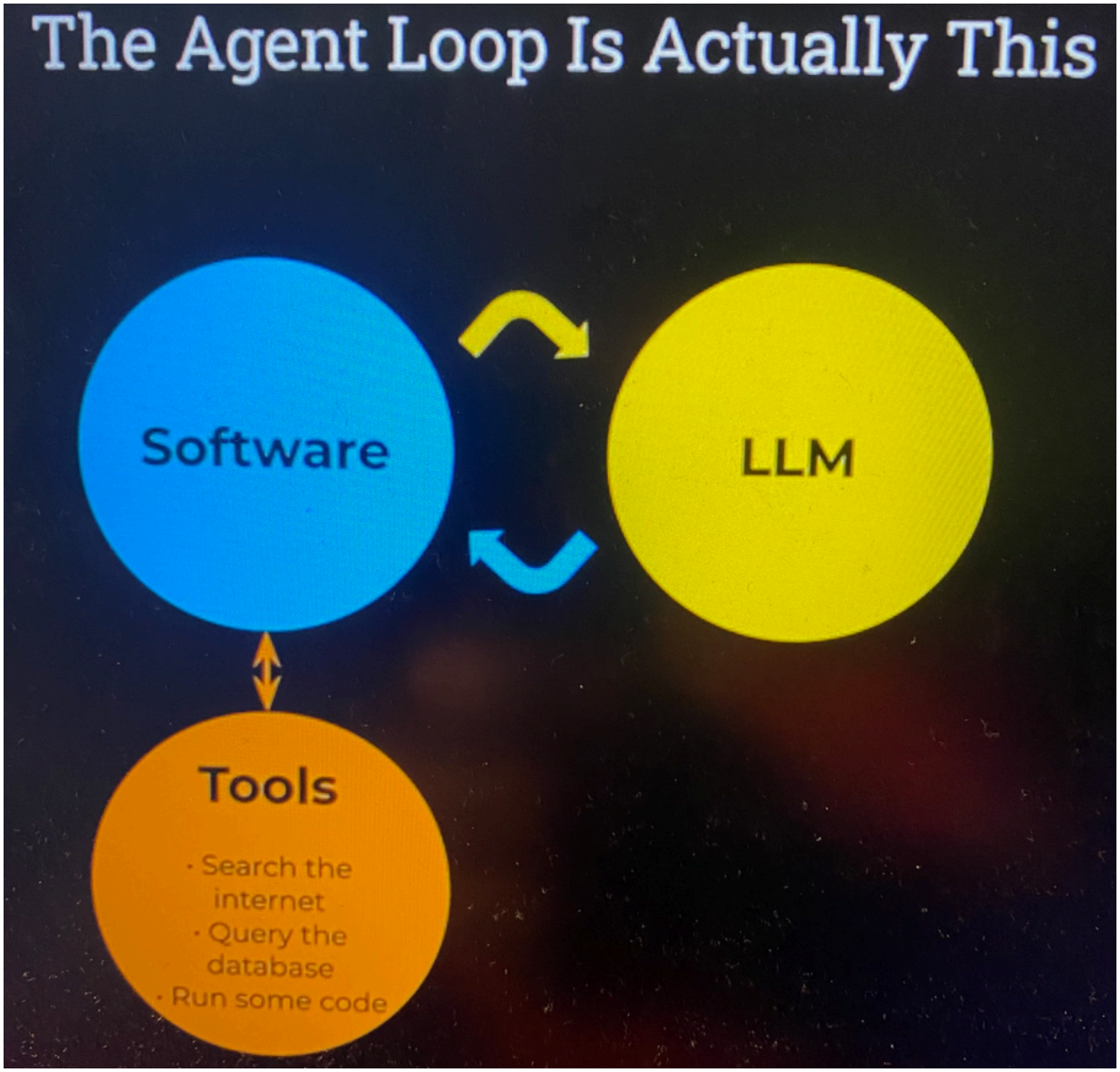
- Tools give LLMs new capabilities
- LLMs by themselves generate text, interpret and spit out tokens, that's all they do **alone**
- LLMs are data science models that take an input sequence of text and they generate the most likely text to follow it based on a complex statistical model
    - They predict the most likely tokens to follow an input sequence of tokens
    - Based on a bunch of matrix calculations
- If we want LLMs to be able to perform other tasks then we have to equip them with tools
    - Look up records in a database
    - Send emails
    - Search the internet
- "Tool calling" is also called "function calling"
- Tools are functions written in code that turned into tools and are fed to the LLM
    - Allow the LLM to perform "stuff"
    - Allow the LLM to run a function on my computer
    - When running the model locally then the function is going to be executed on the localhost
- If LLMs have no access to tools by default, what makes them capable of deciding whether to call a function even when equipped?
    - "I'm not going to generate text, instead I'm going to run this function"
    - It comes down to prompting - we have to explain to the LLM in clear text when and how to use a function
    - The LLM interprets those instructions just like in case of other prompts, there's nothing special about functions in that sense
    - It's a mix of prompts, JSON, if statements, like basic code logic
        - "If the LLM wants us to call a tool, then we will call the tool"
        - …and then we'll call the LLM again with the output from the function call and LLM may come back with another instruction to call a tool, maybe a different one
- Sequence:
    - It's not really the LLM that executed code, reality is even simpler than that - see image below
        - In practice it **feels like** it is the LLM that executes the functions, but **actually** it is our own code that executes the function: the LLM can decide to respond with a plain textual function call like "CALL_SQL_DATABASE" based on all the input text in its context, all previous prompts
    - The user asks a question
    - Our software calls the LLM and says: the user asks this question, what do you want to do?
    - The LLM generates tokens, but in those tokens it can say: tell the user, this is the response, or it can say: don't reply to the user but instead, I want to do an SQL query, please use the SQL tool
    - Then it's our software is the thing that makes the SQL query
    - Then our software calls the LLM again and says: the user said this, we used the tool, here's the result from the tool calling, now what?

    - Then the LLM just continues to generate tokens based on the new input from the function calling
    - The thing that calls the tool is our code, **not** the LLM
    - A vanilla instruction to an LLM can be like this:
        - You are a customer support assistant for an airline. You answer the user's questions. You have a special ability to look up ticket prices. Respond like "GET_TICKET_PRICE: London" to get the ticket price for London. With that, here's the question from the user: "How much does it cost to go to Paris?"
        - The LLM is simply going to response with "GET_TICKET_PRICE: Paris"
        - …which will prompt a function calling in our own software
    - The strength of the LLM is in the ability to interpret the initial instructions and making sense of the user's question to be able to respond with "GET_TICKET_PRICE" followed by the city that the user actually wants to travel to
    - From then on it's up to our code to interpret the LLM's output and call the correct function in a bunch of if statements
    - The LLM generates the pure tool call in text using tokens, the actual execution of the tool is up to us
    - Keep in mind that this is **not perfect**, the initial prompt can be bad, malformed, the LLM can hallucinate and the function that we wanted to be called in the process is **not** called
        - Also, if it happens to respond with the wrong text then our function will not be called, e.g. it responds with "GET_TICKET_PRICES" then tool calling will break down if we don't handle that on our side in code
    - It can also happen that the same tool is called twice or more during the same process, the exact sequence of execution is difficult to predict, you must experiment with the descriptions you provide the tools
    - Another initial prompt that would also demonstrate autonomy is the following:
        - You are exploring a room. You can move in four directions by responding: MOVE NORTH, MOVE EAST, MOVE SOUTH, MOVE WEST to learn more about what's there. Respond only with your choice
        - Then it's up to the LLM to pick one of those four instructions, e.g. "MOVE NORTH": this is called **autonomy**
        - We didn't tell it which one to pick, the LLM made a prediction on its own and made a choice based on whatever criteria it had internally
        - Then if this function is part of a robot, it's up to us to interpret "MOVE NORTH" to make the robot move north by one unit
- Therefore function calling, and especially **successful** function calling is down to good prompting and the LLMs capabilities to interpret the various inputs it gets
- Think of definition #3 of an agent with loops and tools
    - See image further down for a depiction for what tool calling **feels like** vs what it **actually** is

- We call an LLM repeatedly and the LLM might respond with function calls in text with tokens,upon which we execute another tool, respond to the LLM which again may respond with a function call etc. in a loop until we have reached a conclusion
- With good prompts this will be a smooth process but you have to experiment
- With bad prompting and insufficient function calling the loop can get stuck, calling the same function over and over again
    - You have to keep iterating and refining your prompts to make sure that the LLM does what it's supposed to do in all situations
- It's our software that's got the loop, our software that is responsible for making the function calls, our software that must interpret the outcome from the function call and decide whether to continue the loop or decide whether the response is sufficient to fulfil the user's initial prompt, to fulfil a **goal**




The Agent Loop Feels Like This
Software
LLM
Tools
· Search the internet
· Query the database
· Run some code


The Agent Loop Is Actually This
Software
LLM
Tools
· Search the internet
· Query the database
· Run some code

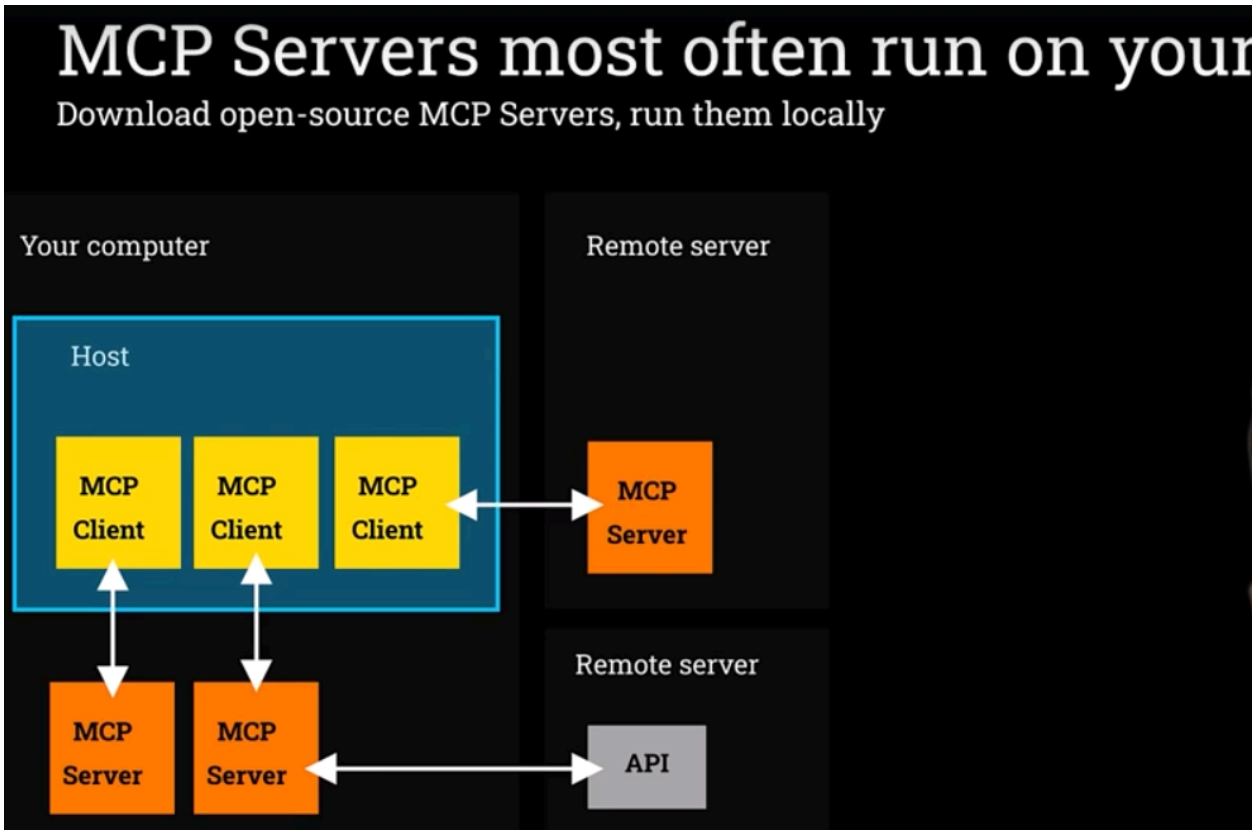
MCP - Model Context Protocol

- What MCP is NOT
    - A framework for building agents
    - It can't be put on the agent framework continuum
    - A fundamental change to how agents work
    - A way to code agents
- What is it then?
    - As it says in the name: a protocol, a standard, a specification, an agreed way to do things, an agreement "we should do it this way" and then life is going to be easier for agentic AI devs
    - A simple way to integrate tools, resources, prompts
        - Things that others have written and that can be integrated into your own agentic workflow and agents
    - A way of integrating disparate agentic workflows (CrewAI with LangChain, OpenAI SDK with CrewAI) using a common interface, integrate any tools that others have written as long as they follow the MCP standard
    - Anthropic: "A USB-C port for AI applications", a universal connector, a plug-and-play, you don't need drivers or any other extras to install, different types of AI apps can share their resources via MCP
    - MCP servers typically include a list of tools that the MCP host can use during its execution
        - These tools come with input parameters, like "get-temperature" may require a city
        - Tools are described in JSON that explains what it means to use a tool and how and when to use it in a way that can be provided to an LLM
        - You can even call the published tools yourself without relying on the LLM to find them
        - MCP servers therefore list tools in a way that LLMs are able to understand, and they themselves can also call other tools and execute actual code on other servers via API calls
        - An MCP server is an interface between an LLM and API calls that actually execute the tools
            - There are cases where all components sit on the localhost such as an MCP server to access your local file system, that won't make any API calls to a remote server
- MCP standard covers tools, resources and prompts
    - …though tools have taken the majority of the integration work, so the big deal with MCP is sharing **tools**
- MCP is really exciting but there are reasons not to be excited as well
    - It's just a standard, a spec, it's not the tools themselves
        - The real innovation is with the tools, not the protocol itself
        - E.g. the Playwright tool by Microsoft which allows us to run browser sessions programmatically via tools in an MCP
        - MCP provides a great way to share and reuse tools but the real innovation comes with the tools behind those MCPs

    - LangChain already has a big Tools ecosystem, so Anthropic wasn't even the first with the idea of sharing tools, MCP is not the only way to get tools
    - Devs can turn their functions into tools regardless of which framework or SDK they use
        - Convert a function into a tool and give it to your agent, so MCP helps you use someone else's tools, not your own tools
- Reasons to love MCP
    - Makes it frictionless to integrate with tools that others have written, easy to connect to them and add them to your agent
    - Wildly successful with the AI community and therefore comes with a huge ecosystem with loads of MCPs published on marketplaces
        - Sometimes it's a downside because there are many similar tools and can be difficult to know which one to take and which one to trust
    - Agreed standards are important and make AI devs' lives easier
        - So successful that Anthropic gave it up to the Agentic AI Foundation which is part of the Linux Foundation
- MCP core concepts
    - Key components which are NOT the same as their counterparts in infrastructure and networks
        - MCP host
            - An LLM app like Claude or Agentic Platform
            - A piece of software that you're running that's going to have an LLM call in it and that we want to hook up tools to
            - An agent harness, ChatGPT, Claude AI
            - An agent that's using and agent framework
            - Claude desktop, Codex, Claude Code or a home-grown agent
        - MCP client
            - Lives inside the host and connects 1:1 to the MCP server
            - "Lives inside": code that runs inside the code that is ChatGPT or your agent
            - Normally there's an MCP client for every set of MCP tools
        - MCP server
            - Provides tools, context and prompts (mostly tools)
            - E.g. tools to operate playwright and drive Chrome
            - The most common scenario is that the MCP server runs on the same machine as the host, i.e. locally, but there are cases where the MCP server runs remotely through a URL
    - The host is the agent you're coding, the client is a piece of software that your agent calls into and the server is another piece of software that's written by a third party and which you're able to connect to
    - E.g. "Fetch" is a popular MCP server with tools to grab a webpage and return it as Markdown
        - So this is an **MCP server**

    - We can configure Claude Desktop (the **host**) to run an **MCP Client** that then launches the Fetch **MCP server** on our computer
        - Equip the host to use a set of tools
    - Agent frameworks take care of creating the MCP client behind the scenes - we just tell them which server to connect to
    - MCP server connection type
        - Installed locally and run completely locally such as an MCP to use the local file system
        - Installed locally but execute API calls that execute the tool(s) remotely
            - The most common scenario
            - The MCP server is only the connecting tissue between the Host and an actual code execution call via an API
        - Installed and executed remotely
            - Can also be called a remote MCP
            - Common in cases where products like Claude desktop are equipped with an MCP server
            - E.g. the Jira MCP server is fully remote so that Claude or ChatGPT can hook up to your Jira account, like a backdoor to access your own Jira account through remote tools instead of writing a bunch of code to integrate with the Jira API yourself
            - Another example: Context 7 which runs a remote server to help LLMs understand information about latest APIs
        - There is also a fourth variant called hosted MCP, or managed MCP, thought not very common
            - Example from OpenAI: everything is located remotely: call a remote MCP which will run an MCP client and an MCP server on “our” end
            - This solution consumes remote infra and typically costs more than the above solutions
            - This also often locks in the user to the provider’s ecosystem with only their own set of LLM models
            - More info here: https://openai.github.io/openai-agents-python/mcp/
- MCP transport mechanism
    - The MCP client and the MCP server communicate using a protocol called MCP that allows it to say “list the tools you can do and call this tool”
    - Technically how does it work?
        - How do the orange and yellow boxes actually connect and communicate?
        - Answer: the transport mechanism and there are multiple options but two ways stand out at present
            - STDIO (standard IO), the simplest one and the one used most frequently
                - The yellow box, the MCP client, launches a separate process on your computer such as a Python program, a Docker container etc and connects to the orange box via

the standard input/output, feels like starting a terminal to communicate with a tool and read its response from the standard output, like calling an exe file from a command line tool (input) and reading its response (output)
    - Often used when MCP is local, though HTTP is also an available option
- HTTP: streamable HTTP, a specialised way of using the HTTP protocol in case we want the information to **flow** between the orange and yellow boxes. There used to be another protocol called SSE but it was replaced by HTTP, although SSE (server-sent events) still exists
    - Typically used in remote MCP server scenarios



- Marketplaces
    - https://glama.ai/mcp
    - https://smithery.ai/
- Making an MCP server
    - Why would you make your own MCP server?
        - If you built a bunch of tools and you want others to be able to reuse them and want to contribute to the “world” then you can make an MCP server for others to load and consume
        - Incorporate tools at a large company so that teams with different tech stacks can share these common tools - instead of creating libs to support

many different languages you can turn the shared tools into an MCP server and it's consistent for anyone at the company to use
        - A company that wants to provide an interface to its clients
        - For educational reasons: we want to understand how and MCP server is made
    - Why you would NOT want to build an MCP server?
        - If it's only for **us** then we can just make tools using the given technique of the framework you're using, any function can be turned into a tool, there's no need for an MCP server
        - MCP servers are often hyped and people create them just to equip their own agents with their in-house tools, that's the wrong way to take - convert your functions into tools and give them to your agents instead
            - In these cases MCP only adds unnecessary technical complexity, it's a code smell, a red herring
            - It introduces an extra layer between your agent and your tools for no reason, it adds to the total execution time
            - → the real innovation is the **tool** in this case, not the MCP server
            - There's no reason to build an MCP server for your own tools that only you use
- MCP context engineering: [The New Skill in AI is Not Prompting, It's Context Engineering]